



OWASP Top 10

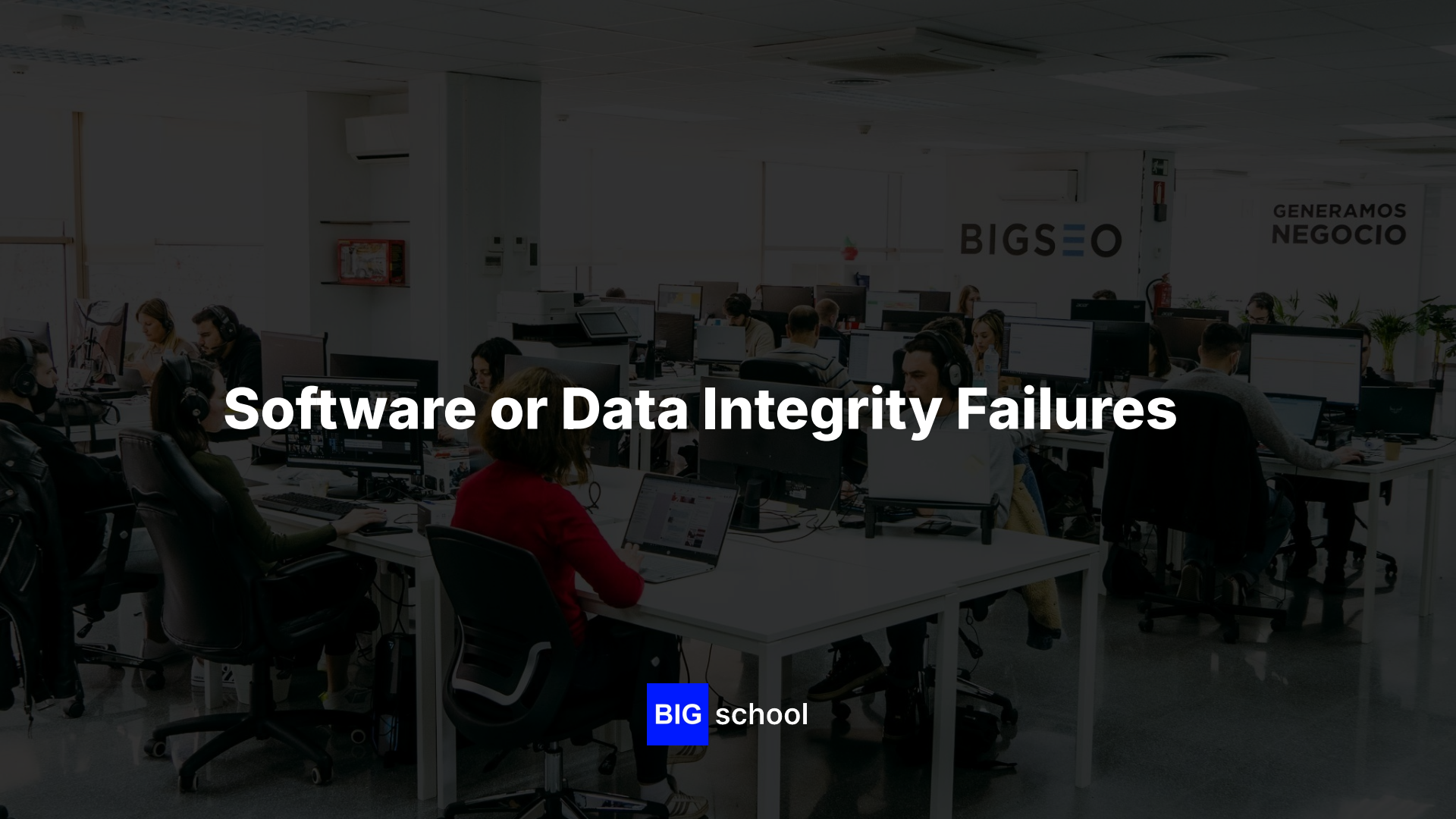
Software or Data Integrity Failures

BIG school

Software or Data Integrity Failures

Índice

- Introducción
- ¿Qué significa esta vulnerabilidad?
- ¿Por qué ocurre esta vulnerabilidad?
- Ejemplos reales y típicos
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Software or Data Integrity Failures

BIG school

Introducción

- **Integridad:** garantizar que software y datos no han sido alterados
- **No basta con código seguro:** hay que verificar lo que consumes y ejecutas
- **Afecta dependencias, pipelines, actualizaciones y deserialización**
- **Objetivo:** confiar solo en lo verificado criptográficamente

¿Qué significa esta vulnerabilidad?

- Falta de verificación de integridad en artefactos, datos o procesos
- Software: paquetes, binarios, imágenes o actualizaciones sin firma/checksum
- Datos: configuraciones, backups o payloads cargados sin validación
- Deserialización insegura: input externo convertido en objetos ejecutables

¿Por qué ocurre esta vulnerabilidad?

- Descarga de dependencias o scripts sin verificar firma o hash
- Pipelines CI/CD que ejecutan código remoto sin control
- Uso de latest o versiones dinámicas sin anclaje
- Deserialización directa de input no confiable
- Backups o configs restauradas sin validación de integridad

Ejemplos reales y típicos

- Paquetes con typosquatting o registros comprometidos (npm, PyPI)
- Actualizaciones automáticas sin firma digital (MITM en deploy)
- Pipelines que descargan scripts sin hash fijo ni allowlist
- Deserialización insegura (pickle, yaml.load, Java ObjectInputStream)
- Configuraciones YAML/JSON remotas cargadas sin autenticación

¿Qué puede hacer un atacante si explota esto?

- Ejecución remota de código mediante deserialización o scripts alterados
- Compromiso persistente inyectando malware en artefactos "legítimos"
- Alteración silenciosa de lógica modificando configs o políticas
- Propagación en cadena si un artefacto comprometido se distribuye
- Evasión de detección: el sistema ejecuta lo alterado como "válido"

¿Cómo detectarlo?

- **Desarrollo:** revisar descargas sin hash/firma, deserialización directa
- **Pipeline:** auditar workflows con permisos excesivos o código remoto
- **Producción:** monitoreo de integridad (AIDE, Tripwire) en binarios críticos
- **Señales:** artefactos sin firma, `yaml.load` inseguro, restauraciones sin checksum

¿Cómo prevenirlo?

- **Firmar y verificar:** GPG, cosign/Sigstore para binarios, contenedores y scripts
- **Anclar versiones:** lockfiles, hashes fijos, evitar latest
- **CI/CD seguro:** firmar artefactos, permisos mínimos, runners efímeros
- **Deserialización segura:** parsers restrictivos (safe_load, allowlist de clases)
- **Integridad de datos:** HMAC/checksums en configs y backups antes de cargar
- **Monitoreo:** alertas de cambios no autorizados en archivos críticos

¿Cómo mitigar si ya estás afectado?

- **Aislar:** detener pipelines y revertir a última versión verificada
- **Invaldar:** eliminar artefactos o configs comprometidas
- **Rotar:** claves de firma, secretos de CI/CD y tokens de despliegue
- **Auditar:** comparar hashes de producción contra baseline conocido
- **Reconstruir:** re-desplegar desde fuentes firmadas con verificación automática

Checklist rápido

- ✓ ¿Paquetes/imágenes/scripts tienen firma o checksum verificable?
- ✓ ¿Se usan versiones fijas/lockfiles en lugar de latest?
- ✓ ¿El pipeline firma artefactos y tiene permisos mínimos?
- ✓ ¿Se evita deserialización directa o se usan parsers seguros?
- ✓ ¿Configs y backups se verifican con hash/HMAC antes de cargar?
- ✓ ¿Hay monitoreo de integridad y alertas de cambios no autorizados?
- ✓ ¿Se rotan claves de firma y secretos de CI/CD periódicamente?

Conclusiones

- La integridad es la garantía de que ejecutas solo lo que decidiste ejecutar
- Verificación criptográfica + trazabilidad + rechazo por defecto = defensa sólida
- No confiar sin verificar: firma, ancla y audita cada artefacto y dato
- Como desarrollador: anclar versiones, validar firmas y usar parsers seguros
- Confiar está bien. Verificar, es seguridad.



¡MUCHAS GRACIAS!

BIG school

BIGSEIO

**GENERAMOS
NEGOCIO**